

操作系统演进样本

从一台没有操作系统的机器开始

每次只解决一个已经出现的问题

这册样稿怎样展开

本册不从“操作系统由哪些模块组成”开始。起点是一台可取指、执行和访问内存，但尚未拥有操作系统的机器。每一章继承上一章结束时的机器，只处理当时已经出现的一项具体困难。

第一章只问一项任务如何开始执行；第二章才允许第二项任务进入机器；第三章再处理任务不愿主动交还处理器的情形。尚未由问题要求的结构不会提前出现。

目录

这册样稿怎样展开	ii
第一章 一台机器怎样只运行一项任务	1
1.1 先把一台能够取到第一条指令的机器接起来	1
1.2 这些器件怎样连接	2
1.3 初始器件的硬件模型、软件模型与抽象	3
1.3.1 时钟与复位：让状态变化具有起点	3
1.3.2 处理器核：把字节解释成指令	3
1.3.3 启动存储：让第一条指令在断电后仍存在	3
1.3.4 RAM：为运行中的变化提供位置	4
1.3.5 地址互连：把一个地址变成唯一目标	4
1.3.6 串行下载接口：把外部信号变成可读取字节	4
1.4 一次取指怎样沿连接完成	4
1.5 通电并不等于任务已经运行	5
1.6 先让任务字节出现在内存中	5
1.7 仅有代码地址为什么还不能开始	6
1.8 任务怎样结束	6
1.9 这一方案确实能完成什么	7
第二章 第二项任务为什么需要常驻协调代码	8
2.1 直接从 A 跳到 B 会丢掉什么	8
2.2 现场保存在哪里	9
2.3 谁有资格保存现场	9
2.4 第一次运行和恢复运行为何不同	9
2.5 最小选择规则怎样形成	10
2.6 一次切换的状态账本	10
2.7 这段常驻代码已经是什么	11
第三章 任务不交还处理器时怎么办	12
3.1 常驻并不等于有机会执行	12
3.2 为什么选择可编程时钟事件	12

3.2.1 新器件接到现有机器的哪里	12
3.2.2 计时器的硬件模型	13
3.2.3 计时器的软件模型	13
3.2.4 计时器提供的抽象	14
3.2.5 处理器异步入口也需要明确三层模型	14
3.3 硬件保存的状态为什么通常不够	15
3.4 事件入口使用哪一份栈	15
3.5 怎样把事件帧并入 B 的任务记录	16
3.6 第一次出现可运行集合	16
3.7 “被唤醒”和“正在运行”尚未出现	17
3.8 一次强制切换的逐时刻状态	17
3.9 本章新增结构的准确边界	17
样稿停在这里	18

第一章 一台机器怎样只运行一项任务

本章要组装的机器。一个处理器核、一片启动存储、一片可读写内存、时钟与复位电路、连接这些器件的地址互连，以及接收任务字节的最小串行接口。没有进程、线程、系统调用、文件和调度器。

1.1 先把一台能够取到第一条指令的机器接起来

单独放置一个处理器并不能运行程序。处理器内部即使具有运算单元和寄存器，也必须持续收到时钟变化，必须在上电后进入已知状态，还必须能从某处读到第一条指令。程序运行后又要保存可变数据，因此只读的启动字节也不能代替全部内存。

本样机采用一组教学化但完整的器件。它们不是现代主板的全部部件，只保留“启动并运行一个任务”真正需要的能力：

器件	此刻必须解决的问题	暂时不承担的职责
时钟与复位电路	让状态按节拍变化，并给出已知启动时刻	不提供日期、定时任务或调度
处理器核	取指、译码、计算并发起内存读写	不知道程序、进程或文件是什么
启动存储	断电后仍保留最早启动指令	不保存任务运行中的栈和变量
RAM	保存运行期会变化的代码副本、数据和栈	不保证断电后内容仍在
地址互连	根据地址把一次请求送到正确器件	不解释地址中保存的业务含义
串行下载接口	把外部任务字节逐个交给处理器读取	不把字节组织成文件系统

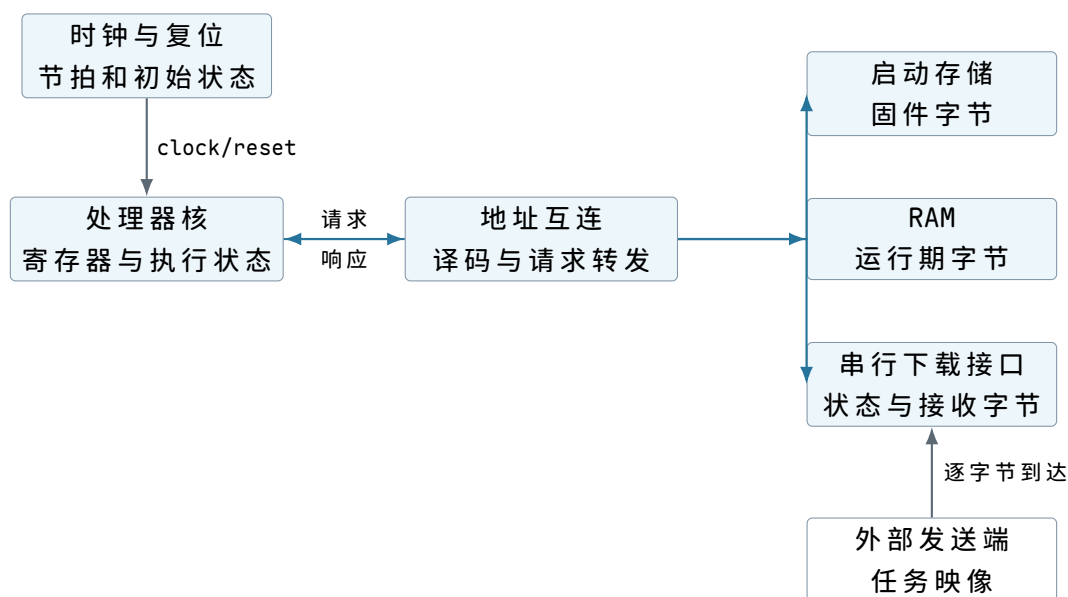
串行接口不是所有裸机都必需。若任务预先固化在启动存储中，机器可以省去它。本书希望在不重新烧写启动固件的情况下更换任务，所以保留一个只能接收字节的最小输入通道。

1.2 这些器件怎样连接

连接不能只用一句“都挂在总线上”概括。处理器访问外部器件时，至少要表达四类信息：

- **地址**：这次请求指向哪个编号位置；
- **方向与宽度**：读取还是写入，一次传送几个字节；
- **写入数据**：写请求准备交给目标的位串；
- **响应**：目标是否完成，以及读请求返回的位串。

这些信息可以由并行导线、片上网络分组或其他协议承载。本样机把它们统称为一次 **总线事务**：处理器发出请求，地址互连选择唯一目标，目标完成后返回响应。这个名称描述一次交互，不要求真实电路必须采用某种特定总线标准。



图示：处理器是当前唯一请求发起者。启动存储、RAM 和串行接口都是地址互连后的目标；串行接口还从机器外部接收字节。图中尚无计时器、中断控制器或操作系统。

互连内部的地址译码器比较请求地址与各器件范围。本样机采用下面的物理地址表；具体数值是教学约定，不是所有处理器的固定标准：

物理地址范围	互连选择的目标	当前用途
0x00000000--0x0000ffff	启动存储	固件与复位后的第一条指令
0x00100000--0x001ffffff	RAM	任务代码、数据、栈和固件工作区
0x40000000	串行状态寄存器	判断是否收到一个新字节
0x40000004	串行数据寄存器	读取刚到达的字节

这里的“寄存器”是串行控制器内部可被处理器寻址的一小块状态，不是处理器通用寄存器。处理器用普通读事务访问 0x40000000 时，互连不会访问 RAM，而会选择串行接口。这种把设备状态纳入地址范围的安排称为内存映射 I/O；它只统一访问形式，不会把设备真的变成内存。

1.3 初始器件的硬件模型、软件模型与抽象

同一个器件可以从三个层次描述。硬件模型说明电路最少保存什么、接收什么、输出什么；软件模型说明程序能通过指令和地址观察什么；抽象则说明后续软件可以稳定依赖哪项能力。三层不能合并，否则容易把电路内部实现误当成软件接口，或把软件约定误写成硬件保证。

1.3.1 时钟与复位：让状态变化具有起点

硬件模型。时钟源周期性产生边沿，处理器等时序电路在规定边沿更新内部状态。复位电路在电源稳定前保持复位有效，释放时使处理器的指令位置和控制状态进入规定初值。

软件模型。普通启动代码不能把“当前经过了多少个时钟边沿”直接当作可用时间。本样机只规定复位释放后，处理器从物理地址 `0x00000000` 取第一条指令。时钟此时没有软件可读寄存器。

抽象。软件可以依赖“指令按体系结构规则逐步执行”和“每次复位从同一入口开始”。它不能依赖真实频率恒定，也不能据此安排若干毫秒后的事件；可编程计时能力要到确有需要时另行加入。

1.3.2 处理器核：把字节解释成指令

硬件模型。处理器保存指令位置、通用寄存器、算术标志和控制状态。它根据指令位置发起取指读事务，译码返回位串，执行运算；遇到装入或存储指令时，再发起数据读写事务。

软件模型。指令集规定哪些位串表示加法、跳转和访存，规定哪些寄存器能被指令读写。栈指针在本样机中使用一个通用寄存器承担；“它是栈顶”属于调用约定，不是存储电路能够识别的特殊数据类型。

抽象。软件得到一条顺序执行的指令流和一组可命名的寄存器。分支可以改变下一条指令位置，读写指令可以访问物理地址。处理器并不自动保证任务只访问分配给自己的范围。

1.3.3 启动存储：让第一条指令在断电后仍存在

硬件模型。启动存储可看作按地址索引的非易失字节阵列。读事务输入地址，输出对应字节；本样机运行期间不允许普通写事务改变它。

软件模型。地址 `0x00000000--0x0000ffff` 可读，内容是预先写入的固件。复位入口落在这个范围内，所以处理器无需先运行别的代码就能取得启动指令。

抽象。软件得到一段跨断电保存、启动时可直接执行的字节序列。它还不是“磁盘”或“文件”；固件中哪段是代码、常量或版本信息，仍由固件自己的布局约定解释。

1.3.4 RAM：为运行中的变化提供位置

硬件模型。RAM 是可读写、断电丢失的存储单元阵列。读事务返回选中单元的内容；写事务根据地址、字节使能和写数据更新单元。

软件模型。程序把 `0x00100000--0x001ffffff` 当作可读写物理地址。处理器不会从 RAM 获得“这里是代码还是栈”的标签；同一字节被怎样解释取决于取指位置和指令。

抽象。软件得到一片可变的字节数组，可用来放代码副本、变量和栈。当前没有缓存一致性、虚拟地址或权限边界，任何处理器存储指令都能改写该范围。

1.3.5 地址互连：把一个地址变成唯一目标

硬件模型。互连接收请求的地址、操作类型和写数据，将地址与范围表比较，只向匹配目标转发请求，再把目标响应送回处理器。无目标匹配或多个目标匹配都属于配置错误；真实机器通常会返回错误响应。

软件模型。软件看到一张物理地址表。对不同范围使用同一类读写指令，却会访问不同器件。一次读只有在收到完成响应后，返回寄存器中的值才有效。

抽象。软件得到统一的物理地址空间：地址足以选择当前机器中的一个目标。这个抽象不等于虚拟内存，也不提供命名、所有权或保护。

1.3.6 串行下载接口：把外部信号变成可读取字节

硬件模型。接收电路把串行线上按时间到达的位组合成一个字节，写入接收寄存器，并设置 `ready=1`。处理器读取数据寄存器后，接口清除 `ready`，等待下一个字节。若旧字节尚未读取而新字节又到达，最小单槽模型会发生溢出。

软件模型。固件反复读取状态地址；看到 `ready` 后读取数据地址，把所得字节写入 RAM。当前接口没有中断和 DMA，所有搬运都由处理器执行：

```
receive_one_byte:
    repeat until read32(0x40000000) has READY
    byte = read8(0x40000004)
    store byte at next RAM address
```

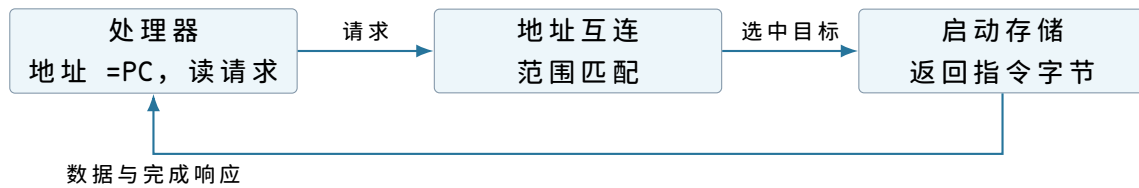
抽象。软件得到一个按到达顺序读取的字节流。它不保证这些字节组成合法任务，不提供文件名、长度或校验；这些规则必须由固件与外部发送端另行约定。

1.4 一次取指怎样沿连接完成

现在可以精确描述复位后的第一条取指，而不再用“处理器读取固件”省略中间步骤：

1. 复位释放后，处理器的指令位置为 `0x00000000`。
2. 处理器发出读请求：地址为 `0x00000000`，方向为读，宽度由指令集规定。
3. 互连发现地址落在启动存储范围，只向启动存储转发请求。
4. 启动存储返回相应字节，互连把完成响应和数据送回处理器。

5. 处理器译码并执行该指令，再按指令语义形成下一条指令位置。



图示：地址决定目标，读写方向决定动作，完成响应决定结果何时有效。总线事务完成后，处理器才拥有可译码的指令字节。

1.5 通电并不等于任务已经运行

处理器能够执行指令，但它不会自行知道“今天要运行哪项任务”。通电复位只会把少量体系结构状态设为规定值，并把下一条取指位置设到一个预先约定的地址。这个地址通常落在固件可用的区域，而不是某个应用程序的开头。

先把机器能够提供的最小事实写清楚：

硬件状态	复位后提供的含义	尚未提供的含义
指令位置	下一次从哪个地址取指	哪项任务应该获得处理器
通用寄存器	若干可参与计算的位串	参数、局部变量或任务身份
内存地址	处理器可访问的字节位置	哪段是代码、数据或栈
设备入口	固件可从外部取得字节	字节属于哪个“文件”

表中右列不是硬件遗漏的缺陷，而是软件必须建立的约定。处理器只认识位模式和地址；“任务”是我们为了描述一组代码、初始数据和运行状态而使用的名称。

1.6 先让任务字节出现在内存中

假设要运行的任务已经被编译成一段机器指令和初始数据。外部发送端先发送一个固定长度头部，再发送任务载荷：

```

download_header:
  payload_bytes
  requested_load_address
  entry_offset
payload:
  machine instructions and initial data
  
```

这只是固件与发送端的传输约定，不是文件格式。固件先检查载荷长度和目标范围，再从串行数据寄存器逐字节读取载荷并写入 RAM。为了避免隐藏关键条件，给出一个具体布局：

物理范围	本次用途	必须保持的约束
------	------	---------

0x100000--0x107fff	任务代码与只读常量	入口地址必须落在已装入代码中
0x108000--0x10bfff	任务可写数据	初值由任务映像给出，其余字节清零
0x10c000--0x10ffff	向下增长的栈	初始栈顶取范围末端，不能越过数据区
0x080000--0x08ffff	固件与装入代码	启动完成前不能被任务映像覆盖

这里还没有内存保护。表格描述的是装入者承诺遵守的布局，不是硬件已经阻止越界。若任务中的错误指令写到 0x080000，固件仍可能被覆盖。

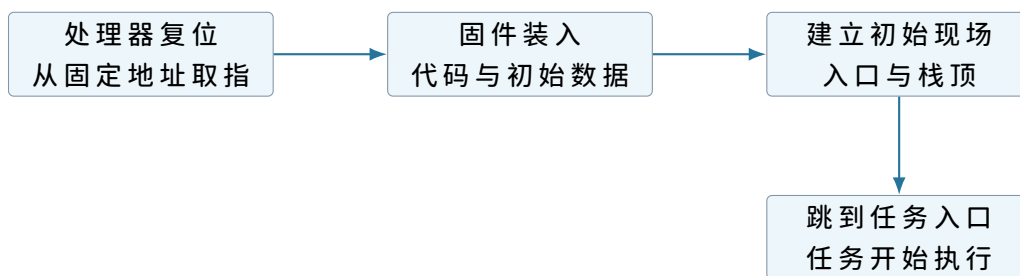
1.7 仅有代码地址为什么还不能开始

把指令写到 0x100000 后直接跳转，仍然缺少栈。普通函数调用要保存返回地址和局部状态；如果栈指针没有确定值，第一次压栈就会写入未知位置。

本次启动至少要形成以下初始现场：

```
instruction pointer = task entry address
stack pointer      = 0x110000
general registers  = documented argument values or zero
interrupt state    = the value required by the startup contract
addressing mode    = the mode used to build the task
```

“现场”在这里还不是操作系统对象。它只是处理器继续执行所需的一组具体状态。入口地址决定取哪条指令，栈指针决定调用与局部数据写到哪里，执行模式决定同一位串如何被解释。



图示：“装入”和“开始执行”是两个时刻。前者改变内存，后者改变处理器的下一条指令位置。

1.8 任务怎样结束

这台机器没有可供任务调用的退出服务。任务入口若执行普通函数返回，栈中也没有一个真实调用者留下的返回地址。最小约定可以要求任务在完成后跳到固件提供的固定返回入口：

```
task_entry:
```

```
compute the requested result  
place result code in agreed register  
jump firmware_return
```

固件返回入口读取结果码，把处理器置于等待状态，或者重新启动同一任务。这里的“返回”完全依赖任务合作；任务必须执行那条跳转。

1.9 这一方案确实能完成什么

到此为止，机器已经能完成一条完整但很窄的路径：

1. 固件选定一个任务映像并复制到约定物理范围；
2. 固件清零未初始化数据，设置入口地址与栈顶；
3. 处理器跳到任务入口；
4. 任务直接使用全部处理器和物理内存；
5. 任务主动跳回固件入口并给出结果。

不应把这台机器描述成已经有了“单任务操作系统”。我们只建立了装入与启动约定；没有常驻任务管理对象，也没有隔离、抢占或统一服务接口。

章末出现的问题。现在有任务 A 和任务 B。运行 A 时，B 的代码放在哪里？A 完成后，谁重新装入 B 并为它建立初始现场？若想暂停 A、先运行 B、以后再继续 A，需要保存的又不只是一个入口地址。下一章只处理这项困难。

第二章 第二项任务为什么需要常驻协调代码

继承的机器。固件能装入一项任务，为它设置入口、栈和初始寄存器；任务完成时主动跳回固定入口。所有地址仍是物理地址，任务仍可访问整台机器。

2.1 直接从 A 跳到 B 会丢掉什么

设 A 正在累加一组数据。下一条指令地址保存在指令位置寄存器中，当前循环计数和部分和位于通用寄存器，函数调用关系位于 A 的栈。此时若把指令位置改成 B 的入口，处理器会立即开始取 B 的指令；但它仍带着 A 的栈顶与寄存器值。

给 B 设置自己的栈和寄存器可以让 B 正确开始，却会覆盖处理器中唯一的一份 A 状态。以后再把指令位置改回 A，并不能恢复 A，因为“不知道回到 A 的哪条指令”只是丢失状态的一部分。

必须保留的状态	丢失后的表现	为什么不能从代码映像重建
继续指令位置	A 从错误位置执行或重新开始	它取决于暂停发生的时刻
栈指针	返回地址与局部变量无法找到	栈深度随已经完成的调用变化
通用寄存器	循环变量、地址和中间结果改变	它们是运行结果，不是初始映像
状态标志	后续条件分支得到不同结果	标志来自暂停前最后一次运算
约定的扩展状态	浮点或向量计算被 B 覆盖	同样属于运行中的中间结果

这组状态可以称为任务的处理器现场。名称出现在问题之后：我们需要一份数据，能够让暂停的任务在未来像没有被中断一样继续，才有必要给这份数据命名。

2.2 现场保存在哪里

不能把 A 的现场继续留在处理器寄存器里，因为 B 就要使用这些寄存器。也不能把所有现场都压到同一栈上，再任意切换栈而不记位置。最小安排是为每项任务分配一块现场记录和一段独立栈：

```
task_state A:
  status
  resume_instruction
  stack_pointer
  general_registers
  arithmetic_flags
  stack_low, stack_high

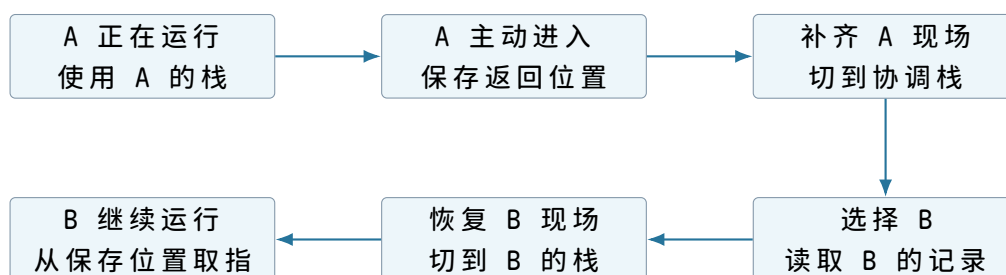
task_state B:
  same fields, different storage
```

`status` 至少区分“尚未开始”“可以继续”“正在运行”和“已经完成”。它不是为了美化结构，而是避免协调代码把一份只有初始入口、尚无暂停现场的记录当作可恢复记录。

2.3 谁有资格保存现场

若 A 自己把寄存器依次写入记录，会遇到一个细节：执行保存代码本身也需要寄存器，而且“保存后的继续地址”应当指向 A 交还处理器之后的位置。最小方案规定一个调用约定：A 像调用函数一样进入协调代码，调用约定先把一部分状态压到 A 的栈，协调代码再补存其余状态并记录当前栈指针。

协调代码必须常驻在不会被任务映像覆盖的内存中。它还需要一个只在选择任务和恢复现场时使用的协调栈，否则恢复 A 的栈之后再运行复杂选择代码，会误把协调者的局部数据写进 A。



图示：切换不是把 A 的代码换成 B 的代码，而是先把处理器唯一的活动现场写回 A 的记录，再把 B 的记录装入处理器。

2.4 第一次运行和恢复运行为何不同

B 第一次获得处理器时，没有“暂停位置”。它的记录由装入者人工构造成一种可恢复形态：继续位置设为 B 的入口，栈指针设为 B 的初始栈顶，其余寄存器按启动约定给值。以后 B 主动交还处理器，记录才被真实运行状态覆盖。

B 的阶段	记录中的继续位置	栈中已经存在的内容
尚未第一次运行	任务入口	人工准备的初始参数或空栈
运行后主动交还	交还入口之后的继续点	B 已经形成的调用链与局部数据
已经完成	不再作为可恢复位置使用	可等待回收或检查结果

统一的恢复代码只读取记录，不必在最后一步区分“第一次”和“后来”。区别已经在记录如何构造、状态字段是否允许恢复中表达。

2.5 最小选择规则怎样形成

当前只有 A 和 B。协调代码可使用一个当前编号：A 交还时若 B 可继续，就选择 B；B 交还时再选择 A。这个规则尚不需要复杂队列，但仍要处理任务完成：

```
choose_next:
    inspect the other task
    if it is ready:
        return that task
    if current task only yielded and remains ready:
        return current task
    otherwise:
        wait for an external event
```

这里第一次出现“选择下一项可运行任务”的职责。由于选择只发生在任务主动进入协调代码后，它仍是合作式安排：协调者能决定交还之后运行谁，却不能决定任务何时交还。

2.6 一次切换的状态账本

时刻	处理器活动现场	A 的内存记录	B 的内存记录
t_0	A 的状态	旧值，不可用于恢复 当前 A	B 的初始或暂停状态
t_1	A 正在执行保存入口	部分字段已更新	不变
t_2	协调代码使用协调栈	A 的完整暂停状态	不变
t_3	正在装入 B 的状态	完整且稳定	被读取，尚未运行
t_4	B 的活动现场	A 可恢复	B 记录成为旧值

任一时刻，正在运行任务的最新寄存器在处理器中，其内存记录只是上一次保存值；暂停任务的最新现场则必须完整位于内存记录和它自己的栈中。说“每项任务都有一份现场”时，需要同时说明活动现场究竟位于处理器还是内存。

2.7 这段常驻代码已经是什么

它不是完整操作系统，却已经具有三个此前不存在的职责：保存当前任务、选择下一任务、恢复所选任务。由于任务运行期间协调代码仍保留在内存中，可以把它称为常驻协调者。以后若加入更多任务，选择规则和记录集合会扩展，但本章不提前设计最终结构。

章末出现的问题。 A 可以约定每处理一批数据就主动交还处理器；B 却可能进入无限循环，或者因为错误永远不调用交还入口。此时协调代码虽然常驻，却没有任何机会执行。下一章只处理“机器怎样在任务不合作时重新得到控制”。

第三章 任务不交还处理器时怎么办

继承的机器。常驻协调者能保存、选择和恢复 A/B；每项任务有独立现场记录与栈。切换入口只能由正在运行的任务主动调用。

3.1 常驻并不等于有机会执行

B 执行下面的指令序列：

```
repeat_forever:
  compute
  jump repeat_forever
```

处理器会不断从 B 的地址取指。协调代码虽然存在于另一段内存，但“存在”不会自动改变指令位置。外部观察者也不能靠在内存里写一个标志解决问题，因为 B 没有读取该标志。

要在 B 不执行交还指令时重新获得控制，机器必须提供一种与当前指令流独立的事件：事件到达后，处理器先保存足以继续 B 的状态，再把指令位置改成预先登记的入口。

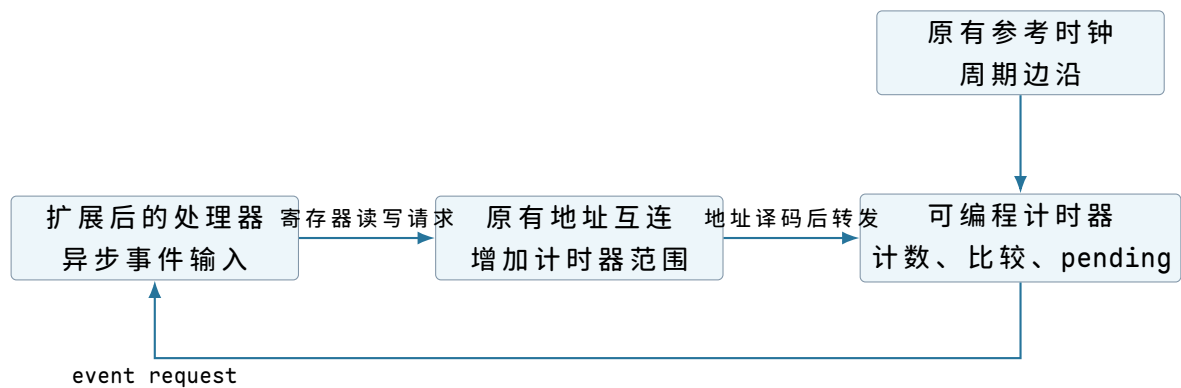
3.2 为什么选择可编程时钟事件

设备完成也能产生外部事件，但机器空闲时未必有设备活动，不能用它保证处理器一定被收回。一个可编程计时器可以在设定时间后发出事件，与 B 是否读内存、是否调用函数无关。

3.2.1 新器件接到现有机器的哪里

第一章的时钟源只能推动电路状态变化，软件不能要求它在未来某个计数值通知处理器。现在增加的计时器是一项独立器件：它接收同一参考时钟，通过地址互连暴露配置寄存器，并用一根事件请求线连接处理器的异步事件输入。

本样机只有一个事件源，所以计时器请求线直接连接处理器，不增加中断控制器。以后事件源增多，才会出现“多根请求线如何编号、屏蔽和路由”的新问题。



图示：总线方向用于配置和查询，事件请求线用于计时器主动通知处理器。软件写过寄存器并不等于事件已经发生；只有计数条件满足后，计时器才置 pending 并拉起请求线。

3.2.2 计时器的硬件模型

计时器内部保存四类最小状态：

内部状态	状态变化	对外作用
counter	每个参考时钟边沿递增	提供单调增加的比较基准
deadline	处理器通过写事务设置	指定下一次到期计数值
enabled	处理器启用或停止计时比较	决定是否检查到期条件
pending	到期时置 1，确认后清 0	为 1 时保持事件请求有效

每个时钟边沿后，若 `enabled=1` 且 `counter >= deadline`，硬件就设置 `pending=1`。请求线保持有效，直到软件确认事件。采用“保持”而不是一个极短脉冲，可以避免处理器暂时关闭事件接收时永久丢失通知。

这个模型没有自动周期。协调代码处理一次到期事件后，写入新的 `deadline`，才形成重复时间片。这样可以先看清“本次到期”和“安排下次到期”是两个动作。

3.2.3 计时器的软件模型

软件不直接访问内部连线，而是通过地址互连看到一组内存映射寄存器：

物理地址与名称	访问方式	软件可见语义
0x40001000 COUNT	只读	返回当前 counter 快照
0x40001008 DEADLINE	读写	设置下一次到期比较值
0x40001010 CONTROL	读写	bit 0 控制 enabled
0x40001014 STATUS/ACK	读/写	读 bit 0 得 pending；写 1 清除 pending

让计时器在 Q 个计数单位后产生一次事件，软件按以下顺序配置：

```

now = read64(COUNT)
write64(DEADLINE, now + Q)
write32(STATUS_ACK, 1)      // clear an earlier pending event
write32(CONTROL, ENABLE)

```

若先启用、后写 deadline，旧 deadline 可能立即满足并产生一次非预期事件。寄存器顺序因此是软件模型的一部分，不只是代码风格。

3.2.4 计时器提供的抽象

上层软件得到“在单调计数达到 deadline 后形成一个保持到确认的事件”。它没有得到以下保证：

- deadline 不是日期，也不自动等于毫秒；
- 到期表示请求已经提出，不表示处理器恰好在该计数值执行入口；
- 事件被屏蔽时，请求可以延迟交付；
- 计时器不会保存任务现场，也不会选择下一项任务。

因此，计时器抽象适合保证协调代码最终重新获得一次入口机会；任务选择仍属于软件。

3.2.5 处理器异步入口也需要明确三层模型

计时器有了请求线，原处理器仍只会顺序取指。机器还要扩展处理器的事件入口能力。

硬件模型。处理器在一条指令完成后检查事件输入。若事件允许且请求有效，处理器保存下一条指令位置、原栈指针和状态标志，切到固定入口栈顶，关闭同类事件嵌套，再把指令位置改为固定事件入口。事件返回指令执行相反的受控恢复。

软件模型。固件把事件入口约定为 `0x00008000`，把入口栈顶约定为 `0x001f0000`。硬件建立的最小帧按顺序包含：

```

event_frame:
    resume_instruction
    interrupted_stack_pointer
    saved_flags

```

入口代码能读取这份帧，并通过确认寄存器清除计时器 pending。普通通用寄存器尚未由硬件保存，必须由入口软件补齐。

抽象。软件得到一个可在普通指令流之外发生的受控入口，以及一份足以返回被打断位置的最小继续状态。该抽象不等于完整上下文切换：它不保存全部寄存器，也不决定事件到达后继续原任务还是恢复另一个任务。

为支持这条路径，硬件至少要明确：

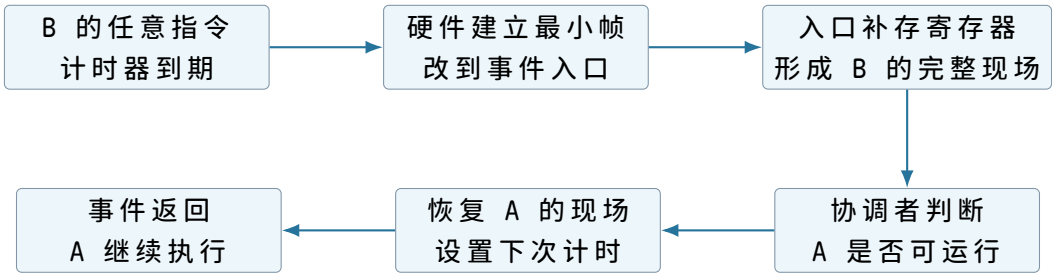
硬件能力	必须提供的事实	软件随后才能做的事
------	---------	-----------

计时器	到期时产生一个可识别事件	规定最长连续运行区间
事件入口表	事件编号对应固定入口地址	让协调代码得到控制
最小现场保存	保存被打断继续位置与必要标志	以后从原指令流继续
事件屏蔽状态	防止入口最早阶段被同类事件反复覆盖	完成一次一致的现场建立
事件返回指令	从规定帧恢复状态与执行级别	回到所选任务

硬件不必替软件决定运行 A 还是 B。它只保证事件发生时能离开 B，并把控制交到一个软件已知入口。选择仍由协调代码完成。

3.3 硬件保存的状态为什么通常不够

事件到达的第一目标是安全进入入口，而不是一次完成所有任务切换。处理器通常只自动保存继续位置、状态标志以及进入和返回所必需的少量字段。通用寄存器若要保持不变，入口软件仍要尽早保存。



图示：时钟事件只强制打开入口机会。完整保存、任务选择和恢复仍是软件协议。

入口不能先使用尚未保存的通用寄存器计算，因为那会覆盖 B 的值。最早几条指令只能使用硬件约定允许破坏的状态，或者把寄存器按固定顺序压入已知安全的栈。

3.4 事件入口使用哪一份栈

B 的栈可能已接近边界，甚至因为错误把栈指针改成无效值。若硬件入口无条件继续使用 B 的栈，保存第一项状态就可能失败。机器需要一个不由普通任务控制的入口栈位置；单核样机可让硬件或最早入口从固定协调状态取得它。

栈	保存的调用关系	所有权边界
A 的任务栈	A 的函数返回地址与局部数据	A 运行或暂停时保留

B 的任务栈	B 的函数返回地址与局部数据	B 运行或暂停时保留
协调入口栈	事件最早保存与选择代码局部状态	普通任务不应改写

入口栈解决的是“如何安全开始保存”，不是自动隔离。由于当前机器仍允许 B 写任意物理地址，B 依然能够故意或错误地覆盖协调入口栈；这个缺陷留到章末。

3.5 怎样把事件帧并入 B 的任务记录

事件帧记录 B 被打断的位置，软件补存通用寄存器。协调者可以让 B 的任务记录指向这份完整帧，而不必把每个字段再复制一次。关键是记录必须说明帧位于哪一段栈、布局采用哪个入口约定。

```
timer_entry:
    hardware has saved resume position and flags
    save remaining registers in fixed order
    current_task.saved_stack = current stack pointer
    current_task.status = ready
    switch to coordinator stack
    next = choose_ready_task()
    restore next.saved_stack
    return_from_event
```

主动交还与时钟打断可以最终汇入相同的“已形成完整现场”边界，但两者最早入口不同。主动交还遵守普通调用约定；时钟可以发生在任意指令，必须按异步入口约定保存所有可能被后续代码破坏的状态。

3.6 第一次出现可运行集合

只有两项任务时也需要区分“暂停但能继续”和“已经完成”。协调者不应把完成的任务重新装入处理器。可以维护一个很小的可运行集合：

任务状态	是否参与选择	状态改变的入口
ready	是	被时钟打断或主动交还后
running	否，当前已占用处理器	协调者恢复该任务时
finished	否	任务进入完成入口时
not started	可先构造初始现场再加入	装入者完成代码、数据和栈准备时

选择规则可以是 A、B 轮流。每次恢复前重新设置计时器，给所选任务一个有限运行区间。时间区间结束并不表示任务完成，只表示协调者获得一次重新选择的机会。

3.7 “被唤醒”和“正在运行”尚未出现

本章任务只做计算，没有等待设备或条件；ready 已足够表达“具有运行资格”。以后加入等待后，某项任务即使没有完成也可能暂时不应进入选择集合，届时才需要额外状态和唤醒协议。现在提前画完整生命周期只会让尚无问题来源的概念进入样机。

3.8 一次强制切换的逐时刻状态

时刻	控制流位置	B 的最新现场	A 的最新现场
t_0	B 的循环	处理器中	A 的内存记录与栈
t_1	硬件事件入口	最小字段在事件帧， 其余仍在寄存器	不变
t_2	软件入口完成保存	B 的完整帧与栈	不变
t_3	协调者选择 A	稳定，可供以后恢复	正在被读取
t_4	事件返回到 A	B 的内存现场	处理器中

事件返回的目标不必是刚才被打断的 B。硬件要求返回帧格式正确；软件可以在返回前换成 A 的已保存帧。这个能力使时钟入口成为抢占切换点。

3.9 本章新增结构的准确边界

这一阶段只新增：

- 可编程计时器及其事件入口；
- 不依赖普通任务栈的最早入口状态；
- 能容纳异步打断的完整任务现场格式；
- ready/running/finished 三类选择状态；
- 每次恢复前重新设置时间预算的轮流选择规则。

它没有新增文件、用户态、系统调用、虚拟地址、权限或设备服务。所谓“最小抢占调度”在此只表示：协调者能在任务未主动交还时获得入口，并从可运行集合选择另一项任务。

章末出现的问题。 B 已经不能永久独占处理器，却仍能执行一条写指令覆盖 A 的栈、A 的代码、协调者记录或时钟入口。时间上的轮流使用已经建立，空间上的互不破坏仍未建立。下一次演进应从这次真实覆盖失败出发，再决定硬件与软件需要增加什么；本样稿在此停止。

样稿停在这里

此时机器已经能装入多项任务，并能依靠时钟事件收回处理器。然而任务仍共享物理地址，任何任务都能覆盖协调代码或其他任务。这是下一次演进应处理的问题；本样稿暂不提前给出答案，等待对叙事粒度与技术深度进行评审后再继续。